

A. Proof of Diffusion Loss for Full Conditional Action Distribution

Most treatments of diffusion models have been studied primarily in the context of single-modality distributions, such as those over image pixels [3], [34], [52]. This formulation has been directly adopted by the robotics community [4], [53], [54], leading to the optimization objective shown in Equation 9.

$$\mathcal{L}_t(\theta) = \mathbb{E}_{(\mathbf{x}_0, \mathbf{y}) \sim q(\mathbf{x}_0, \mathbf{y}), \epsilon_0 \sim \mathcal{N}(0, \mathcal{I})} [\|\epsilon_0 - \hat{\epsilon}_\theta(\mathbf{x}_t, \mathbf{y}, t)\|_2^2] \quad (9)$$

We formally show that a conditional diffusion process as defined by Dhariwal and Nichol [39] results in Equation 9 being a maximizer of the reweighted variational lower bound [3] on the log-likelihood of the conditional data distribution $\log q(\mathbf{x}|\mathbf{y})$. This implies that by incorporating the observational modalities into the conditioning process, diffusion policies [4] learn the reverse transition kernel for action conditioned on all the modalities, as shown in the following proof. We adopt a slightly different notation from the main paper where we use q for the forward and the reverse diffusion process, and the resulting marginals.

Lemma 1: The diffusion loss function $\mathcal{L}_t(\theta)$ as defined in Equation 9, in expectation over the time-steps $1 \leq t \leq T$, maximizes the reweighted variational lower bound [3] on the log-likelihood of the conditional data distribution $\log q(\mathbf{x}|\mathbf{y})$, under a Markovian noising process $\hat{q}(\mathbf{x}_t|\mathbf{x}_{t-1})$ and the conditional reverse transition kernel as $\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})$.

Here, we derive the diffusion loss function for the conditional distribution $p(\mathbf{x}|\mathbf{y})$ instead of only $p(\mathbf{x})$. A parallel derivation for conditional variational auto-encoders can be found in Doersch [55]. Following Dhariwal and Nichol [39], we start with a conditional Markovian noising forward process $\hat{q}$ similar to $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1 - \alpha_t)\mathcal{I})$, and define the following:

$$\hat{q}(\mathbf{x}_0) := q(\mathbf{x}_0) \quad (10)$$

$$\hat{q}(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{y}) := q(\mathbf{x}_{t+1}|\mathbf{x}_t) \quad (11)$$

$$\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y}) := \prod_{t=1}^T \hat{q}(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{y}) \quad (12)$$

We now reproduce some results that will be used later in the derivation of diffusion loss for conditional distributions. Dhariwal and Nichol [39] also show that

$$\hat{q}(\mathbf{y}|\mathbf{x}_t, \mathbf{x}_{t+1}) = \hat{q}(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{y}) \frac{\hat{q}(\mathbf{y}|\mathbf{x}_t)}{\hat{q}(\mathbf{x}_{t+1}|\mathbf{x}_t)} \quad (13)$$

$$= \hat{q}(\mathbf{x}_{t+1}|\mathbf{x}_t) \frac{\hat{q}(\mathbf{y}|\mathbf{x}_t)}{\hat{q}(\mathbf{x}_{t+1}|\mathbf{x}_t)} \quad (14)$$

$$= \hat{q}(\mathbf{y}|\mathbf{x}_t) \quad (15)$$

Moreover, the unconditional reverse transition kernels can be shown to be equal using Bayes theorem, given Equations 10 and 11: $\hat{q}(\mathbf{x}_t|\mathbf{x}_{t+1}) = q(\mathbf{x}_t|\mathbf{x}_{t+1})$. Dhariwal and Nichol [39] use the result from Equation 15 to show the following

for conditional reverse transition kernels.

$$\hat{q}(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}) = \frac{\hat{q}(\mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{y})}{\hat{q}(\mathbf{x}_{t+1}, \mathbf{y})} \quad (16)$$

$$= \frac{\hat{q}(\mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{y})}{\hat{q}(\mathbf{y}|\mathbf{x}_{t+1})\hat{q}(\mathbf{x}_{t+1})} \quad (17)$$

$$= \frac{\hat{q}(\mathbf{x}_t|\mathbf{x}_{t+1})\hat{q}(\mathbf{y}|\mathbf{x}_t, \mathbf{x}_{t+1})\hat{q}(\mathbf{x}_{t+1})}{\hat{q}(\mathbf{y}|\mathbf{x}_{t+1})\hat{q}(\mathbf{x}_{t+1})} \quad (18)$$

$$= \frac{\hat{q}(\mathbf{x}_t|\mathbf{x}_{t+1})\hat{q}(\mathbf{y}|\mathbf{x}_t, \mathbf{x}_{t+1})}{\hat{q}(\mathbf{y}|\mathbf{x}_{t+1})} \quad (19)$$

$$= \frac{q(\mathbf{x}_t|\mathbf{x}_{t+1})\hat{q}(\mathbf{y}|\mathbf{x}_t)}{\hat{q}(\mathbf{y}|\mathbf{x}_{t+1})} \quad (20)$$

Further, we can show the following using Equations 11 and 12 and the Markovian noising process. It states that the joint distribution of the noised samples conditioned on $\mathbf{y}$ and $\mathbf{x}_0$ are the same for both $\hat{q}$ and q .

$$\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y}) = \prod_{t=1}^T \hat{q}(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{y}) \quad (21)$$

$$= \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) \quad (22)$$

$$= q(\mathbf{x}_{1:T}|\mathbf{x}_0) \quad (23)$$

We adapt the derivation of diffusion loss from Luo [56] to work with conditional distributions by maximizing the log-likelihood of the conditional data distribution $\log p(\mathbf{x}|\mathbf{y})$ leading to evidence lower bound (ELBO).

$$\log p(\mathbf{x}|\mathbf{y}) = \log \int p(\mathbf{x}_{0:T}|\mathbf{y}) d\mathbf{x}_{1:T} \quad (24)$$

$$= \log \int \frac{p(\mathbf{x}_{0:T}|\mathbf{y})\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})}{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} d\mathbf{x}_{1:T} \quad (25)$$

$$= \log \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\frac{p(\mathbf{x}_{0:T}|\mathbf{y})}{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \right] \quad (26)$$

$$\geq \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_{0:T}|\mathbf{y})}{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \right] \quad (27)$$

The ELBO can be further simplified as follows

$$\begin{aligned}
\log p(\mathbf{x}|\mathbf{y}) &\geq \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_{0:T}|\mathbf{y})}{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y}) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\prod_{t=1}^T \hat{q}(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{y})} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y}) p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})}{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})} \right. \\
&\quad \left. + \log \prod_{t=2}^T \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\hat{q}(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0, \mathbf{y})} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y}) p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})}{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})} \right. \\
&\quad \left. + \log \prod_{t=2}^T \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\frac{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y}) \hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})}{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_0, \mathbf{y})}} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y}) p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})}{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})} \right. \\
&\quad \left. + \log \frac{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})}{\hat{q}(\mathbf{x}_T|\mathbf{x}_0, \mathbf{y})} + \log \prod_{t=2}^T \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y}) p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})}{\hat{q}(\mathbf{x}_T|\mathbf{x}_0, \mathbf{y})} \right. \\
&\quad \left. + \sum_{t=2}^T \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} \right] \\
&= \mathbb{E}_{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})] \\
&\quad + \mathbb{E}_{\hat{q}(\mathbf{x}_T|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p(\mathbf{x}_T|\mathbf{y})}{\hat{q}(\mathbf{x}_T|\mathbf{x}_0, \mathbf{y})} \right] \\
&\quad + \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t, \mathbf{x}_{t-1}|\mathbf{x}_0, \mathbf{y})} \left[\log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})}{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} \right] \\
&= \underbrace{\mathbb{E}_{\hat{q}(\mathbf{x}_1|\mathbf{x}_0, \mathbf{y})} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1, \mathbf{y})]}_{\text{reconstruction term}} \\
&\quad - \underbrace{D_{\text{KL}}(\hat{q}(\mathbf{x}_T|\mathbf{x}_0, \mathbf{y}) \parallel p(\mathbf{x}_T|\mathbf{y}))}_{\text{prior matching term}} \\
&\quad - \sum_{t=2}^T \underbrace{\mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} [D_{\text{KL}}(\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y}) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y}))]}_{\text{denoising matching term}}
\end{aligned} \tag{28}$$

The reconstruction term is ignored for training [3], [56], and the prior matching term does not have any trainable parameters. We further simplify the denoising matching term using Equation 20 further conditioned on $\mathbf{x}_0$.

$$\begin{aligned}
&- \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} [D_{\text{KL}}(\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y}) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y}))] \\
&= - \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} [\mathbb{E}_{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} [\log \hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y}) \\
&\quad - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})]] \\
&= - \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} [\mathbb{E}_{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} [\log \hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \\
&\quad + \log \frac{\hat{q}(\mathbf{y}|\mathbf{x}_{t-1}, \mathbf{x}_0)}{\hat{q}(\mathbf{y}|\mathbf{x}_t, \mathbf{x}_0)} - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})]] \\
&= - \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} [D_{\text{KL}}(\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y}))] \\
&\quad - \sum_{t=2}^T \mathbb{E}_{\hat{q}(\mathbf{x}_t|\mathbf{x}_0, \mathbf{y})} \left[\mathbb{E}_{\hat{q}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0, \mathbf{y})} \left[\log \frac{\hat{q}(\mathbf{y}|\mathbf{x}_{t-1}, \mathbf{x}_0)}{\hat{q}(\mathbf{y}|\mathbf{x}_t, \mathbf{x}_0)} \right] \right]
\end{aligned} \tag{29}$$

Note that the expectation is taken over a distribution independent of $\mathbf{y}$, since $\hat{q}(\mathbf{x}_{1:T}|\mathbf{x}_0, \mathbf{y}) = q(\mathbf{x}_{1:T}|\mathbf{x}_0)$, as shown in Equation 23. It is easy to see that the first term in the resulting expression is the KL divergence between the model parameterized with the condition $\mathbf{y}$ and the unconditional reverse transition kernel, leading to the popularly used diffusion loss of Equation 9. However, an additional term is introduced for the conditional diffusion process. This minimizes the difference in the likelihood of the labels between consecutive denoising steps. However, since it does not have trainable parameters, we ignore it.

In Equation 9, $\epsilon_\theta(\mathbf{x}_t, \mathbf{y}, t)$ arises from the reparametrization of the reverse transition kernel $q_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y}^{1:M})$. In this work, we argue that learning the full conditional directly is restrictive in several aspects of robot learning. Firstly, it necessitates the joint collection of the robot action and all observational modalities. Secondly, the model is vulnerable to even small distribution shifts in *any* modality. These shifts require a prohibitively large amount of data to address when the observation modalities are high-dimensional. Finally, among the multiple observation modalities it is hard to pinpoint the level of each mode's task dependent influence with limited data.

B. Proof for FDP Loss

Theorem 2: Explicit score matching for $\mathbf{s}_\phi(\mathbf{x}_t, \mathbf{y}^{1:M})$ in Equation 3 is equivalent to the objective $\mathcal{J}_{\alpha_t}^{res}(\phi)$:

$$\mathbb{E}_{p_{\alpha_t}(\mathbf{x}, \mathbf{x}_t, \mathbf{y}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha_t}(\mathbf{x}_t | \mathbf{x}) - \begin{matrix} \mathbf{s}^*(\mathbf{x}_t, \mathbf{y}^{1:k}) \\ -\mathbf{s}_\phi(\mathbf{x}_t, \mathbf{y}^{1:M}) \end{matrix} \right\|_2^2 \right]$$

Here $\mathbf{s}^*(\mathbf{x}_t, \mathbf{y}^{1:k})$ is the the frozen optimal score model for $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{y}^{1:k})$, approximated using a learned model $\pi_{\text{base}} : \mathbf{s}_\theta(\mathbf{x}_t, \mathbf{y}^{1:k})$ in practice.

Chao et al. [37] in their insightful work for score-based models, show that the following two losses differ only by a constant, where the $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{y}}$ notation is introduced to indicate Gaussian noised variables, with variances α and τ respectively.

$$\mathcal{D}_F(p_\phi(\tilde{\mathbf{y}}|\tilde{\mathbf{x}}) \| p_{\alpha,\tau}(\tilde{\mathbf{y}}|\tilde{\mathbf{x}})) = \mathbb{E}_{p_{\alpha,\tau}(\tilde{\mathbf{x}}, \tilde{\mathbf{y}})} \left[\frac{1}{2} \left\| \begin{matrix} \nabla_{\tilde{\mathbf{x}}} \log p_\phi(\tilde{\mathbf{y}}|\tilde{\mathbf{x}}) \\ -\nabla_{\tilde{\mathbf{x}}} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}|\tilde{\mathbf{x}}) \end{matrix} \right\|_2^2 \right] \quad (31)$$

$$\mathcal{L}_{DLSM}(\phi) = \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}, \tilde{\mathbf{x}}, \mathbf{y}, \tilde{\mathbf{y}})} \left[\frac{1}{2} \left\| \begin{matrix} \nabla_{\tilde{\mathbf{x}}} \log p_\phi(\tilde{\mathbf{y}}|\tilde{\mathbf{x}}) + \nabla_{\tilde{\mathbf{x}}} \log p_\theta(\tilde{\mathbf{x}}) \\ -\nabla_{\tilde{\mathbf{x}}} \log p_\alpha(\tilde{\mathbf{x}}|\mathbf{x}) \end{matrix} \right\|_2^2 \right] \quad (32)$$

We extend their proof for diffusion models and multiple conditionals below. Here $\mathbf{x}_t$ is noised with the forward transition kernel $p_{\tilde{\alpha}_t}(\mathbf{x}_t | \mathbf{x}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\tilde{\alpha}_t} \mathbf{x}, (1 - \tilde{\alpha}_t)I)$. Explicit Score Matching loss between the model and the true score of the classifier can be further expanded as:

$$\mathcal{D}_F(p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \| p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k})) \quad (33)$$

$$= \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) - \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] \quad (34)$$

$$= \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\rangle \right] \quad (35)$$

$$= \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}, \tilde{\mathbf{y}}^{k+1:M}) \right\rangle \right] - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\rangle \right] \quad (36)$$

$$= \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) \right\rangle \right] - \underbrace{\mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}, \tilde{\mathbf{y}}^{k+1:M}) \right\rangle \right]}_{\text{Term 1}} \quad (37)$$

Simplifying the Term 1 further:

$$\begin{aligned} & - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:M}) \right\rangle \right] \\ & = - \int_{\mathbf{x}_t} \int_{\tilde{\mathbf{y}}^{1:M}} p_\tau(\tilde{\mathbf{y}}^{1:M}) p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:M}) \left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \frac{\nabla_{\mathbf{x}_t} p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:M})}{p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:M})} \right\rangle d\tilde{\mathbf{y}}^{1:M} d\mathbf{x}_t \\ & = - \int_{\mathbf{x}_t} \int_{\tilde{\mathbf{y}}^{1:M}} p_\tau(\tilde{\mathbf{y}}^{1:M}) \left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \int_{\mathbf{x}_0} p_{0,\tau}(\mathbf{x}_0 | \tilde{\mathbf{y}}^{1:M}) p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0, \tilde{\mathbf{y}}^{1:M}) d\mathbf{x}_0 \right\rangle d\tilde{\mathbf{y}}^{1:M} d\mathbf{x}_t \\ & = - \int_{\mathbf{x}_t} \int_{\tilde{\mathbf{y}}^{1:M}} p_\tau(\tilde{\mathbf{y}}^{1:M}) \left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \int_{\mathbf{x}_0} \int_{\mathbf{y}^{1:M}} p_{0,\tau}(\mathbf{x}_0 | \tilde{\mathbf{y}}^{1:M}) p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0, \tilde{\mathbf{y}}^{1:M}, \mathbf{y}^{1:M}) p(\mathbf{y}^{1:M} | \mathbf{x}_0, \tilde{\mathbf{y}}^{1:M}) d\mathbf{y}^{1:M} d\mathbf{x}_0 \right\rangle d\tilde{\mathbf{y}}^{1:M} d\mathbf{x}_t \\ & = - \int_{\mathbf{x}_t} \int_{\tilde{\mathbf{y}}^{1:M}} \int_{\mathbf{x}_0} \int_{\mathbf{y}^{1:M}} p_\tau(\mathbf{x}_0, \mathbf{x}_t, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M}) \left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0, \tilde{\mathbf{y}}^{1:M}, \mathbf{y}^{1:M}) \right\rangle d\mathbf{y}^{1:M} d\mathbf{x}_0 d\tilde{\mathbf{y}}^{1:M} d\mathbf{x}_t \\ & = - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_0, \mathbf{x}_t, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0) \right\rangle \right] \end{aligned}$$

Plugging this back into Equation 37, we get:

$$\begin{aligned} & \mathcal{D}_F(p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \| p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k})) \\ & = \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) \right\rangle \right] - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_0, \mathbf{x}_t, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0) \right\rangle \right] \quad (38) \end{aligned}$$

Here, $\mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right]$ is a constant. Further, adding the constant $\mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:k})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) - \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0) \right\|_2^2 \right]$ to Equation 38, we get:

$$\begin{aligned} & \mathcal{D}_F(p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \| p_{\alpha,\tau}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k})) \\ & = \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) \right\|_2^2 \right] + \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) \right\rangle \right] - \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_0, \mathbf{x}_t, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M})} \left[\left\langle \nabla_{\mathbf{x}_t} \log p_\phi(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}), \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \mathbf{x}_0) \right\rangle \right] \end{aligned}$$

$$+ \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:k})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\alpha,\tau}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) - \nabla_{\mathbf{x}_t} \log p_{\alpha}(\mathbf{x}_t | \mathbf{x}_0) \right\|_2^2 \right] + C \quad (39)$$

$$= \mathbb{E}_{p_{\alpha,\tau}(\mathbf{x}, \mathbf{x}_t, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M})} \left[\frac{1}{2} \left\| \nabla_{\mathbf{x}_t} \log p_{\phi}(\tilde{\mathbf{y}}^{k+1:M} | \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}) + \nabla_{\mathbf{x}_t} \log p_{\theta}(\mathbf{x}_t | \tilde{\mathbf{y}}^{1:k}) - \nabla_{\mathbf{x}_t} \log p_{\alpha}(\mathbf{x}_t | \mathbf{x}) \right\|_2^2 \right] + C \quad (40)$$

Simplifying $\nabla_{\mathbf{x}_t} \log p_{\alpha}(\mathbf{x}_t | \mathbf{x})$ to $-\epsilon_0 / \sqrt{1 - \bar{\alpha}_t}$, where $\epsilon_0 \sim \mathcal{N}(0, \mathcal{I})$, and taking the $(1 - \bar{\alpha}_t)$ times the DSM objective [33], we obtain the simplified diffusion loss:

$$\mathcal{L}_{res}^t(\phi) = \mathbb{E}_{p_{\tau}(\mathbf{x}, \mathbf{y}^{1:M}, \tilde{\mathbf{y}}^{1:M})} \mathbb{E}_{\epsilon_0 \sim \mathcal{N}(0, \mathcal{I})} \left[\left\| \epsilon_0 - \epsilon_{\theta}(\mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}, t) + \hat{\epsilon}_{\phi}(\tilde{\mathbf{y}}^{k+1:M}, \mathbf{x}_t, \tilde{\mathbf{y}}^{1:k}, t) \right\|_2^2 \right] + C \quad (41)$$

C. Architecture and Implementation Details

All transformer-based models are trained over 2000 epochs for visual tasks and 3000 epochs for low-dimensional tasks. UNet [42] is trained over 3000 epochs for visual tasks and 5000 epochs for low-dimensional tasks. We train models on visual tasks with a batch size of 64, and low-dimensional tasks with a batch size of 256. All models are trained on NVIDIA A5000 or A40 GPUs, with training times ranging from 6 to 12 hours depending on model size and the number of camera inputs. Our current implementations support an action prediction latency of ~ 50 ms for DP-DiT, ~ 100 ms for UNet [4] and output composition of models as shown in Figure 2 [b] and ~ 150 ms for FDP model shown in [c].

DP-DiT. We use DiT-S (~ 33 M parameters) [43] as the base architecture, with 12 layers, 6 heads and a hidden dimension of 6. Peebles et al. [43] specifically show that the conditioning using AdaLn-Zero outperforms other forms of conditioning such as in-context and cross-attention for image generation. However, we observe slightly stronger performance when the weights for the final layer of AdaLn are initialized with a Gaussian. We use different untrained ResNet-18 (~ 12 M parameters) [57] encoders for each camera and also encode proprioception using a separate encoder. All encoded conditionals across the observation horizon are concatenated before using AdaLn. The model size conditioned on the input images from 2 cameras is ~ 56 M parameters. All DiT models are trained using a learning rate of $1e-4$ and a weight decay of $1e-3$. We also perform exponential moving average (EMA) to reduce the variance in training. The same DiT backbone is used for low-dimensional, visual, and point-cloud tasks. We use the 100-step DDPM [3] noise scheduler suggested by CHi et al. [4] implemented using HuggingFace Diffusers [58]. Sampling is performed using 8-step DDIM [52].

DP-UNet. We use the 1D-UNet [42] implementation from Chi et al. [4]. UNet is trained using a learning rate of $1e-4$ and a weight decay of $1e-6$. Although the parameter count of the DiT model does not vary significantly with increasing context length due to self-attention, that is not the case with UNet. We use a relatively smaller UNet for low-dimensional

tasks and a UNet with a larger channel width for visual tasks. For an observation horizon of 3 and an action horizon of 15 for visuomotor tasks, the parameter count of UNet increases to (~ 336 M), not including the ResNet weights. UNet uses FiLM [59] layers for conditioning on a single embedding, which is built for different visual inputs and proprioception similar to DP-DiT.

FDP. We experiment with several implementations of FDP in this paper, as shown in Figures 2 [b] and [c]. The simplest implementation shown in [b] simply adds the output of the base and the residual model, with the base kept frozen. The architectures of these models are exactly identical to DP-DiT, except that they are conditioned on different modalities. For Figure 2 [c] used to present the results in the paper, the architecture of the base model is the same as that of DP-DiT. However, the residual model is designed similarly to the ViT [44] architecture. Since we do not denoise the inputs, we encode and pass all the inputs across the observation horizon through self-attention. We condition them on noisy actions using AdaLn. The images are encoded using patch embedding, where we keep the patch size equal to the size of the image to reduce the number of parameters. Crucially, we apply a zero layer on the block outputs of the residual model that are added to the corresponding blocks of the base model. We implement two variants of the zero-layer: a zero-initialized convolutional layer and a zero-initialized linear layer. For the convolutional layer, π_{base} learned on proprioception is of ~ 30 M parameters and the residual model π_{res} with two camera image inputs is of ~ 55 M parameters. However, the linear zero layer bloats the residual model’s size to ~ 290 M. Other hyperparameters such as the learning rate, weight decay, and the noise schedule are the same as DP-DiT.

D. Experimental Setup

RLBench. We train visuomotor policies in RLBench using a multi-camera setup that records 96×96 RGB images, with joint positions as the action modality. We experiment on RLBench in both two and five camera setups (wrist, front, overhead, right-shoulder, and left-shoulder), and with an observation horizon of 2, and an action horizon of 16. Along with modifications to add distractors and color variations (Figure 3), we also create a custom `block-pick` environment in three different sizes as shown in Figure 11.

Adroit. The Adroit benchmark comprises high-dimensional hand manipulation tasks performed using a 24-DoF anthropomorphic hand (see Figure 10). It includes four tasks—*Door*, *Hammer*, *Pen*, and *Relocate*—that demand fine motor control and complex object interaction. We modify the success condition of the *Hammer* task, requiring the nail to be within a distance of 0.2 (instead of 0.1) from the board. Each Adroit task is represented by a task-specific low-dimensional state vector. We use an observation horizon of 3 and an action horizon of 15 across all Adroit experiments.

Robomimic. The dataset [47] provides low-dimensional state observations and uses an action space defined as the

TABLE IV

ENVIRONMENT SPECIFICATIONS—OBS/ACT HORIZON USES ON/HM. FOR RL BENCH, M3L AND REAL-WORLD, ENV-OBSERVATION DEPENDS ON #CAMERAS (1/3/5) AND 512-D EMBEDDINGS.

Suite	Task	Env. Obs. Dim.	Rob. Obs. Dim.	Action Dim.	Max. Len.	# Train Demos	Obs/Act Horizon
Robomimic	Lift	10	9	7	400	10/50/100	o1h10
	Can	14	9	7	400	10/50/100	o1h10
	Square	14	9	7	400	10/50/100	o1h10
	Toolhang	44	9	7	700	10/50/100	o1h10
Adroit	Door	12	27	28	475	22	o2h16
	Hammer	13	33	26	475	22	o2h16
	Pen	21	24	24	475	22	o2h16
	Relocate	9	30	30	475	22	o2h16
RLBench	Various	—	8	8	300/600	10/50/100	o3h15
M3L	Insertion	—	0	3	300	100/200	o1h1
Real-World	Various	—	9	9	400	50	o3h15

change in end-effector position and orientation (axis-angle). The benchmark includes four tasks: *Lift*, *Can*, *Square*, and *Toolhang*, with the latter two requiring higher precision. Following Chi et al. [4], we use an observation horizon of 1 and an action horizon of 10 for all Robomimic experiments.

M3L. We use one 64×64 image input and two 32×32 tactile inputs to the model from the two gripper fingers holding the peg. The rotation of the end effector is frozen, and ΔXYZ is the chosen action space. Various shapes of pegs are provided by the environment, which can be inserted into the respective holes present in differently shaped blocks. In our paper, we show results for models trained on 2 pegs independently, and also on the whole assortment of pegs and blocks.

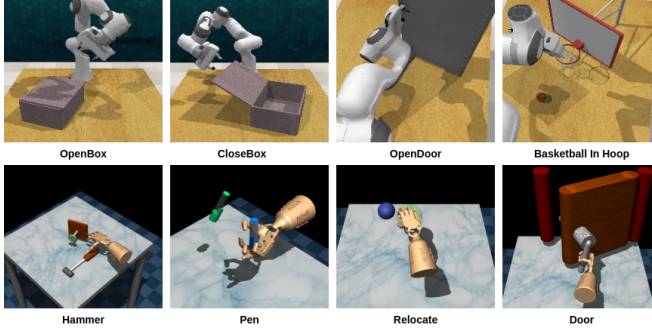


Fig. 10. Selected tasks from RLBench and tasks from Adroit are shown.

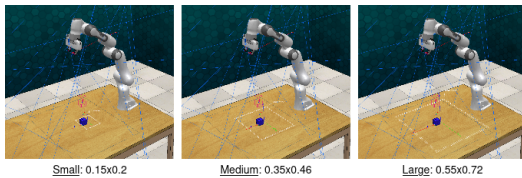


Fig. 11. Task design for block pick with different scales of variation (dim in meters).

Real Robot Experiments. The task domains used in our

real-world experiments as shown in Figures 12 are described below:

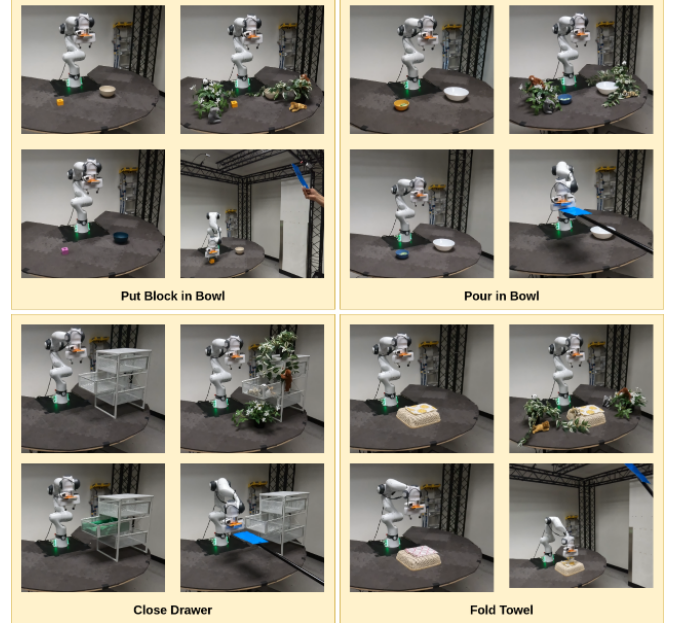


Fig. 12. Tasks considered for the real robot experiments. In clockwise direction: original task, with distractors, with camera occlusions, and with color changes.

- *Close Drawer*: Close the cabinet of an open drawer. We vary the drawer’s placement angle and position relative to the robot within a range of 10° and 15 cm, respectively. This is a relatively simple task where the robot must close the drawer by pushing it with its end-effector.
- *Put Block in Bowl*: Pick up a block and place it inside a nearby bowl. The positions of both the block and the bowl are varied within a 15 cm range relative to the robot. This task assesses the policy’s ability to perform precise pick-and-place actions.
- *Pour in Bowl*: Pick up a cup and pour its contents into a nearby bowl. The positions of the cup and the bowl are varied within a 15 cm range relative to the robot. This task evaluates the policy’s effectiveness in operating near joint limits.
- *Fold Towel*: Fold a kitchen towel placed on a compliant surface. The towel’s position is varied within a 5 cm range relative to the robot. This task evaluates the policy’s capability in deformable object manipulation.

We used ROS1 Noetic for robot software development. For data collection, we used a 3D Connexion SpaceMouse Pro to set end-effector velocity targets, which were executed on the Franka robot using a differential inverse kinematics controller. Time-synchronized joint positions and camera images were recorded at 30Hz for each demonstration and later post-processed by downsampling to 10Hz for policy training. During rollout, we employed a joint position controller to sequentially execute short-horizon trajectory predictions from the policy. We allowed a trajectory length of 400 steps for each task in all our real-world robot experiments. With

a horizon length of 16, this resulted in 25 policy inference steps per task.

Robot Safety Check. We implemented a safety check in our robot software to prevent potential damage to the robot during environment variation experiments. This was particularly necessary for DP, which often generated high-jerk joint targets in out-of-distribution scenarios. For each joint command, we ensured that the target was within a threshold Euclidean distance from the current joint state, i.e., $\|j_{\text{target}} - j_{\text{current}}\| \leq 0.1$. If this condition was violated, policy execution was immediately halted and the rollout was considered a failure.

REFERENCES

- [1] B. Wahn and P. König, “Is attentional resource allocation across sensory modalities task-dependent?” *Advances in cognitive psychology*, vol. 13, no. 1, p. 83, 2017.
- [2] M. O. Ernst and M. S. Banks, “Humans integrate visual and haptic information in a statistically optimal fashion,” *Nature*, vol. 415, no. 6870, pp. 429–433, 2002.
- [3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [4] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [7] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascali, M. Russi, A. Marafioti *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [8] W. Liu, J. Mao, J. Hsu, T. Hermans, A. Garg, and J. Wu, “Composable part-based manipulation,” *arXiv preprint arXiv:2405.05876*, 2024.
- [9] Z. Yang, J. Mao, Y. Du, J. Wu, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “Compositional diffusion-based continuous constraint solvers,” *arXiv preprint arXiv:2309.00966*, 2023.
- [10] L. Wang, J. Zhao, Y. Du, E. H. Adelson, and R. Tedrake, “Poco: Policy composition from and for heterogeneous robot learning,” *arXiv preprint arXiv:2402.02511*, 2024.
- [11] Y. Luo, U. A. Mishra, Y. Du, and D. Xu, “Generative trajectory stitching through diffusion composition,” *arXiv preprint arXiv:2503.05153*, 2025.
- [12] U. A. Mishra, S. Xue, Y. Chen, and D. Xu, “Generative skill chaining: Long-horizon skill planning with diffusion models,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2905–2925.
- [13] U. A. Mishra, Y. Chen, and D. Xu, “Generative factor chaining: Coordinated manipulation with diffusion-based factor graph,” in *ICRA 2024 Workshop {textemdash} Back to the Future: Robot Learning Going Probabilistic*, 2024.
- [14] T. Yu, T. Xiao, A. Stone, J. Tompson, A. Brohan, S. Wang, J. Singh, C. Tan, J. Peralta, B. Ichter *et al.*, “Scaling robot learning with semantically imagined experience,” *arXiv preprint arXiv:2302.11550*, 2023.
- [15] Z. Chen, S. Kiani, A. Gupta, and V. Kumar, “Genaug: Retargeting behaviors to unseen situations via generative augmentation,” *arXiv preprint arXiv:2302.06671*, 2023.
- [16] M. Du, S. Nair, D. Sadigh, and C. Finn, “Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets,” *arXiv preprint arXiv:2304.08742*, 2023.
- [17] M. Memmel, J. Berg, B. Chen, A. Gupta, and J. Francis, “Strap: Robot sub-trajectory retrieval for augmented policy learning,” *arXiv preprint arXiv:2412.15182*, 2024.
- [18] L.-H. Lin, Y. Cui, A. Xie, T. Hua, and D. Sadigh, “Flowretrieval: Flow-guided data retrieval for few-shot imitation learning,” *arXiv preprint arXiv:2408.16944*, 2024.
- [19] M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid, “Residual reinforcement learning from demonstrations,” *arXiv preprint arXiv:2106.08050*, 2021.
- [20] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, “Policy decorator: Model-agnostic online refinement for large policy model,” *arXiv preprint arXiv:2412.13630*, 2024.
- [21] L. Dodeja, K. Schmeckpeper, S. Vats, T. Weng, M. Jia, G. Konidaris, and S. Tellex, “Accelerating residual reinforcement learning with uncertainty estimation,” *arXiv preprint arXiv:2506.17564*, 2025.
- [22] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, “From imitation to refinement-residual rl for precise assembly,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 01–08.
- [23] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, “Transic: Sim-to-real policy transfer by learning from online correction,” *arXiv preprint arXiv:2405.10315*, 2024.
- [24] Y.-C. Li, F. Zhang, W. Qiu, L. Yuan, C. Jia, Z. Zhang, Y. Yu, and B. An, “Q-adapter: Customizing pre-trained llms to new preferences with forgetting mitigation,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.03856>
- [25] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, “Lora: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [26] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.05543>
- [27] Z. Liu, J. Zhang, K. Asadi, Y. Liu, D. Zhao, S. Sabach, and R. Fakoor, “Tail: Task-specific adapters for imitation learning with large pre-trained models,” *arXiv preprint arXiv:2310.05905*, 2023.
- [28] W. Gu, S. Kondepudi, L. Huang, and N. Gopalan, “Continual skill and task learning via dialogue,” *arXiv preprint arXiv:2409.03166*, 2024.
- [29] M. Sharma, C. Fantacci, Y. Zhou, S. Koppula, N. Heess, J. Scholz, and Y. Aytar, “Lossless adaptation of pretrained vision models for robotic manipulation,” *arXiv preprint arXiv:2304.06600*, 2023.
- [30] J. Wen, Y. Zhu, J. Li, M. Zhu, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, “Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation,” 2025. [Online]. Available: <https://arxiv.org/abs/2409.12514>
- [31] X. Liu, Y. Zhou, F. Weigend, S. Sonawani, S. Ikemoto, and H. B. Amor, “Diff-control: A stateful diffusion-based policy for imitation learning,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 7453–7460.
- [32] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” 2015.
- [33] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” *arXiv preprint arXiv:2011.13456*, 2020.
- [34] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” 2020.
- [35] A. Hyvärinen and P. Dayan, “Estimation of non-normalized statistical models by score matching,” *Journal of Machine Learning Research*, vol. 6, no. 4, 2005.
- [36] P. Vincent, “A connection between score matching and denoising autoencoders,” *Neural computation*, vol. 23, no. 7, pp. 1661–1674, 2011.
- [37] C.-H. Chao, W.-F. Sun, B.-W. Cheng, Y.-C. Lo, C.-C. Chang, Y.-L. Liu, Y.-L. Chang, C.-P. Chen, and C.-Y. Lee, “Denoising likelihood score matching for conditional score-based data generation,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.14206>
- [38] G. O. Roberts and R. L. Tweedie, “Exponential convergence of langevin distributions and their discrete approximations,” 1996.
- [39] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” *Advances in neural information processing systems*, vol. 34, pp. 8780–8794, 2021.
- [40] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [41] Y. Du, C. Durkan, R. Strudel, J. B. Tenenbaum, S. Dieleman, R. Fergus, J. Sohl-Dickstein, A. Doucet, and W. Grathwohl, “Reduce, reuse, recycle: Compositional generation with energy-based diffusion models and mcmc,” 2023.

- [42] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical image computing and computer-assisted intervention-MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.
- [43] W. Peebles and S. Xie, "Scalable diffusion models with transformers," 2023. [Online]. Available: <https://arxiv.org/abs/2212.09748>
- [44] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [45] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark & learning environment," *CoRR*, vol. abs/1909.12271, 2019. [Online]. Available: <http://arxiv.org/abs/1909.12271>
- [46] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," *arXiv preprint arXiv:1709.10087*, 2017.
- [47] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," *arXiv preprint arXiv:2108.03298*, 2021.
- [48] C. Sferazza, Y. Seo, H. Liu, Y. Lee, and P. Abbeel, "The power of the senses: Generalizable manipulation from vision and touch through masked multimodal learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 9698–9705.
- [49] J. Ho and T. Salimans, "Classifier-free diffusion guidance," 2022. [Online]. Available: <https://arxiv.org/abs/2207.12598>
- [50] Z. Zhao, S. Haldar, J. Cui, L. Pinto, and R. Bhirangi, "Touch begins where vision ends: Generalizable policies for contact-rich manipulation," 2025. [Online]. Available: <https://arxiv.org/abs/2506.13762>
- [51] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations," *arXiv preprint arXiv:2403.03954*, 2024.
- [52] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2022. [Online]. Available: <https://arxiv.org/abs/2010.02502>
- [53] M. Reuss, M. Li, X. Jia, and R. Lioutikov, "Goal-conditioned imitation learning using score-based diffusion policies," *arXiv preprint arXiv:2304.02532*, 2023.
- [54] X. Liu, K. Y. Ma, C. Gao, and M. Z. Shou, "Diffusion models in robotics: A survey," 2025.
- [55] C. Doersch, "Tutorial on variational autoencoders," *arXiv preprint arXiv:1606.05908*, 2016.
- [56] C. Luo, "Understanding diffusion models: A unified perspective," 2022. [Online]. Available: <https://arxiv.org/abs/2208.11970>
- [57] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [58] P. von Platen, S. Patil, A. Lozhkov, P. Cuenca, N. Lambert, K. Rasul, M. Davaadorj, D. Nair, S. Paul, W. Berman, Y. Xu, S. Liu, and T. Wolf, "Diffusers: State-of-the-art diffusion models," <https://github.com/huggingface/diffusers>, 2022.
- [59] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. C. Courville, "Film: Visual reasoning with a general conditioning layer," in *AAAI*, 2018.